# Optimal Deterministic Rendezvous in Labeled Lines

Yann Bourreau [1], Ananth Narayanan [1], and Alexandre Nolin [2]

[1]CISPA Helmholtz Center for Information Security, Saarbrücken, Germany
[2]SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris, Palaiseau, France

arXiv:2505.04564v2 [cs.DC] 13 Jan 2026

**Abstract**

In a *rendezvous* task, a set of mobile agents initially dispersed in a network have to gather at an arbitrary common site. We consider the rendezvous problem on the infinite labeled line, with 2 initially asleep agents, without communication, and a synchronous notion of time. Each node on the line is labeled with a unique positive integer. The initial distance between the two agents is denoted by $D$. Time is divided into rounds and measured from the moment an agent first wakes up. We denote by $\tau$ the delay between the two agents' wake up times. If awake in a given round $T$, an agent at a node $v$ has three options: stay at the node $v$, take port 0, or take port 1. If it decides to stay, the agent will still be at node $v$ in round $T+1$. Otherwise, it will be at one of the two neighbors of $v$ on the infinite line, depending on the port it chose. The agents achieve rendezvous in $T$ rounds if they are at the same node in round $T$. We aim for a deterministic algorithm for this problem.

The problem was recently considered by Miller and Pelc [MP25, Distributed Computing 2025]. With $\ell_{\max}$ the largest label of the two starting nodes, they showed that no algorithm can guarantee rendezvous in $o(D \log^* \ell_{\max})$ rounds. The lower bound follows from a connection with the LOCAL model of distributed computing, and holds even if the agents are guaranteed simultaneous wake-up ($\tau = 0$) and are told their initial distance $D$. Miller and Pelc also gave an algorithm of optimal matching complexity $O(D \log^* \ell_{\max})$ when the agents know $D$, but only obtained the higher bound of $O(D^2 (\log^* \ell_{\max})^3)$ when $D$ is unknown to the agents.

In this paper, we improve this second complexity to a tight $O(D \log^* \ell_{\max})$, closing the gap between the best known lower and upper bounds. In fact, our algorithm achieves rendezvous in $O(D \log^* \ell_{\min})$ rounds, where $\ell_{\min}$ is the smallest label within distance $O(D)$ of the two starting positions. We obtain this result by having the agents compute sparse subsets of the nodes to gather at (formally, ruling sets over the line), as well as some general observations about the setting of rendezvous on labeled graphs.

# Contents

**1 Introduction** **3**
- 1.1 Our Results . . . 3
- 1.2 Technical Overview . . . 4
- 1.3 Related Work . . . 6
- 1.4 Organization of the Paper . . . 7

**2 Preliminaries** **8**
- 2.1 Notation . . . 8
- 2.2 Computational Models . . . 8
- 2.3 Useful Subroutines . . . 10

**3 Our Main Methods** **10**
- 3.1 Ensuring Rendezvous from Crossing: CarefulWalk . . . 10
- 3.2 Computing Ruling Sets on Paths . . . 12
- 3.3 Computing Colored Ruling Sets on Paths, and Stopping Early . . . 14

**4 Our Optimal Rendezvous Algorithm** **16**
- 4.1 The Algorithm . . . 16
- 4.2 Analysis Overview . . . 17
- 4.3 First Argument: Out-of-Sync Rendezvous . . . 18
- 4.4 Second Argument: In-Sync Rendezvous . . . 19
- 4.5 Full Analysis . . . 21

**5 Corollary for Finite Graphs** **22**

# 1 Introduction

A classic task for a set of autonomous mobile agents is to meet at some location. The agents might want to gather for a number of reasons: to exchange some resources, to initiate a long journey together, or to assemble in a specific shape. With only two agents, the problem is known as *rendezvous* [AG03].

We consider a fully discrete version of this problem: time is divided into rounds, and in each round, each agent is located at a node of a graph $G$. Furthermore, we assume that each node $v$ of this graph has a unique label $\ell(v) \in \mathbb{N}^*$, and that the graph $G$ inhabited by the agents is an infinite line. The agents wake up at possibly different rounds. Once awake, in each round, an agent can stay at its current location for one round or move to a node neighboring its current location. Rendezvous is achieved if the agents are at the same node in some round. In particular, it does not suffice that the agents are in adjacent nodes, or cross the same edge in opposite directions: they lack a communication or sensing method to detect these situations. The initial distance between the two agents is denoted $D$, and $\tau$ denotes the delay between the two wake-up times. The two agents execute the same algorithm, also known as the *symmetric* setting. We more formally describe our model in Section 2.

## 1.1 Our Results

We give an optimal deterministic algorithm for the rendezvous problem on the infinite labeled line.

**Theorem 1.1.** *There is a deterministic algorithm for solving rendezvous on the infinite labeled line in $O(D \log^* \ell_{\min})$ rounds from the wake-up time of the first agent, where $D$ is the initial distance between the two agents and $\ell_{\min}$ is the smallest label within distance $O(D)$ from the agents' starting locations.*

Theorem 1.1 improves upon a prior algorithm of complexity $O(D^2(\log^* \ell_{\max})^3)$ by Miller and Pelc [MP25], where $\ell_{\max}$ is the largest label of the two starting nodes of the agents. To a lesser extent, we also improve upon their $O(D \log^* \ell_{\max})$ algorithm that assumes that the agents know their initial distance $D$, by replacing their dependency in $\ell_{\max}$ by one in $\ell_{\min}$.

The optimality of our algorithm follows from a lower bound from the same paper by Miller and Pelc [MP25]. They proved that any algorithm running in $o(D \log^* \ell_{\max})$ rounds would imply an $o(\log^* n)$ algorithm for 3-coloring paths and cycles of $\text{poly}(n)$ nodes in the LOCAL model. A seminal result by Linial states that no such $o(\log^* n)$ algorithm exists [Lin92]. Note that our algorithm, while of complexity $o(D \log^* \ell_{\max})$ if considering an infinite family of instances where $\ell_{\min} \in o(\ell_{\max})$, is compatible with this lower bound, as families of instances where $\ell_{\min} \in \Theta(\ell_{\max})$ can be constructed. That is, our algorithm is only of complexity $o(D \log^* \ell_{\max})$ on specific instances, not in general[1].

Our algorithm for the infinite line immediately implies a similar result for finite paths and cycles.

**Corollary 1.2.** *There is a deterministic algorithm for solving rendezvous on finite labeled paths and cycles in $O(\min(n, D \log^* \ell_{\min}))$ rounds from the wake-up time of the first agent, where $D$ is the initial distance between the two agents, $\ell_{\min}$ is the smallest label within distance $O(D)$ from the agents' starting locations, and $n$ is the number of nodes in the graph.*

Like our algorithm for the infinite line, this improves upon previous results of complexity $O(\min(n, D^2(\log^* \ell_{\max})^3))$ and $O(\min(n, D \log^* \ell_{\max}))$ by Miller and Pelc [MP25].

[1] Throughout this paper and as was the case in the paper by Miller and Pelc, we take a slightly non-standard definition of the $\log^*$ function guaranteeing that $\log^*(x) \geq 1$ for all $x$. This is to clarify that our $O(D \log^* \ell_{\min})$ algorithm does not finish in 0 rounds when $\ell_{\min} = 1$, since $\log^*(1) = 0$ with the most common definition of $\log^*$. See the Preliminaries (Section 2).

## 1.2 Technical Overview

**Rendezvous and treasure hunt.** To get some intuition on how rendezvous algorithms typically work, it is instructive to consider one of the simplest variants of the problem: the asymmetric setting with simultaneous startup. Asymmetry means that each agent can execute its own strategy, or equivalently, one agent receives label 1 while the other receives label 2. Simultaneous startup means that the two agents wake up in the same round.

These assumptions allow for the following strategy: one agent goes through successive phases exploring increasingly bigger intervals centered on its initial node, while the other agent waits at its initial node. Having the active agent explore intervals doubling in size between phases, the strategy achieves rendezvous in $O(D)$ rounds, which is optimal up to the hidden constant and lower order terms. Rendezvous reduces to *treasure hunt* in that scenario.

While this algorithm cannot be used as is in harder settings, our goal will be to somehow reduce to this simple strategy. In the symmetric setting but with access to randomness, for example, the agents can flip coins to decide whether to search or stay put during some time intervals. In all likelihood, the agents should regularly sample opposite roles. Similarly, if the agents are given distinct labels (but importantly, not restricted to the set $\{1, 2\}$), distinct behaviors can still be constructed from the given identifiers. The agents might for example alternate searching and inactive phases according to the binary representations of their labels. More generally, we need the agents to *break symmetry* between themselves, so that at some point during the execution, one of the two agents does a search while the other agent stays idle. Given that one agent's idle phase overlaps sufficiently with the other agent's searching phase, the two agents will achieve rendezvous.

**How knowledge of $D$ helps.** Having the agents know the initial distance $D$ between them drastically reduces the space of possibilities. For instance, from the perspective of either agent, there are only two possibilities for the starting location of the other agent: the two nodes at distance exactly $D$ from its own initial node. If each agent spends its first $3D$ rounds to go to these two nodes, the first awake agent always finds the other agent in these $3D$ rounds, granted that the delay $\tau$ between the agents' wake-up times is at least $3D$. Not meeting the other agent in these initial $3D$ rounds thus immediately implies a bound on the delay, $\tau < 3D$.

On the infinite labeled line, consider now the subset of nodes whose distances to the starting location of one of the agents are divisible by $D$. The resulting set of nodes is the same whether we measure the distances from either of the two starting locations, as well as contains them. Connecting each member of this set to the two other members closest to it on the original line, we obtain another infinite labeled line. A possible strategy for the two agents to break symmetry is to agree on a coloring of this sub-line, and to act according to the colors of their starting nodes. As the agents perform searching phases interspersed with waiting phases according to these colors, knowing $D$ allows the agents to limit their phases to intervals of $O(D)$ rounds.

**Coloring an infinite line.** Without additional assumptions, the agents cannot truly agree on a coloring of the whole sub-line. Indeed, the agents can only base the coloring of the sub-line on the labels of the nodes of the infinite line. In order for the agents to be able to compute the color of a node, said color should only depend on a finite set of labels. As the agents cannot explore more than $T$ nodes in $T$ rounds, for the sake of speed, the colors given to the agents' starting nodes should ideally be computable from labels in relative proximity to them.

At a high level, we want a mapping from some universe of possible local neighborhoods to colors, such that if applying this mapping to the infinite line, a valid coloring is obtained. This idea of constructing a mapping between local neighborhoods and a node's local output almost corresponds to the LOCAL model of distributed computing. A key difference, however, is that the standard LOCAL model assumes a finite graph.

Still, it is easy to adapt a seminal LOCAL algorithm for 3-coloring paths and cycles to obtain a useful mapping. To do so, the algorithm is transformed into an *early-stopping* one, which has nodes terminate earlier depending on their given labels. While the classic LOCAL algorithm for 3-coloring paths and cycles completes in $O(\log^* n)$ rounds, assuming that the nodes are given unique identifiers between 1 and $\text{poly}(n)$, the early-stopping version instead guarantees that a node $v$ of label $\ell(v)$ permanently commits to a color after $O(\log^* \ell(v))$ rounds. From the point of view of mapping local views to colors, this means that there exists a mapping that infers the color of a node of label $\ell(v)$ from its distance-$O(\log^* \ell(v))$-neighborhood. Used on the infinite sub-line obtained by taking one out of every $D$ nodes, this allows the two agents to assign distinct colors to their starting nodes after $O(D \log^* \ell_{\max})$ rounds of exploration, where $\ell_{\max}$ is the largest of the starting locations' labels. This roughly summarizes the approach of Miller and Pelc [MP25].

**Limitations of the regular sub-line approach.** The approach sketched in previous paragraphs is very dependent on the assumption that the agents have knowledge of $D$, as it has the agents essentially ignore all but a specific $1/D$ fraction of the line. If the agents run the same algorithm with a wrong "guess" for $D$, the two agents are effectively not synchronizing in any way, as the sub-lines they each construct are only the same if the agents' guess for $D$ is a divisor of the actual distance $D$. Even when their guess divides $D$, the agents' starting nodes are only adjacent in the sub-line if the guess is exactly $D$.

As a result, to tackle the setting without knowledge of $D$ by using the previous algorithm with different guesses for $D$ basically requires to test *every possible value* for $D$. This is true even if the agents are given an upper bound on $D$. This difficulty is the source of the $O(D^2)$ dependency in the earlier algorithm by Miller and Pelc [MP25].

**Limitations of a purely coloring-based approach.** For simplicity, let us assume for now that the agents are given an upper bound on $D$. In the paper, we will see that handling a fully unknown $D$ can be handled relatively easily by essentially considering geometrically increasing bounds on $D$. As a first try, let us aim for the agents to agree on a common coloring of the line that ensures that their starting locations receive distinct colors.

With only an upper bound on $D$, the agents now need to break symmetry with $2D$ nodes instead of just 2. Guaranteeing that any two nodes within distance $D$ from one another are colored differently cannot be done with 3 colors. Pushing forward with this approach, one could color the line with $O(D)$ colors. But now, the increased number of colors necessitates to design the same number of distinct behaviors of alternating searching and waiting phases guaranteeing rendezvous. This higher number of behaviors is likely to lead to a $D \log D$ dependency in the distance, similar to how in anonymous networks with labeled agents, larger labels lead to higher complexity in this fashion[2]. This would be far from the $\Omega(D \log^* \ell)$ lower bound.

**Our solution: ruling set nodes as local landmarks.** To reduce the number of behaviors we have to design, and thus, improve on the complexity that would follow from an approach based on computing a distance-$D$ coloring, we have the agents agree on a $1/\Theta(D)$ fraction of the nodes that act as common milestones, or landmarks. One way to understand our algorithm is that we have the agents *build their common sparse sub-line* instead of receiving it from the get-go as when they know $D$. Formally, the agents compute a *ruling set* over the line: a set of nodes such that every two of its members are at least at some minimum distance $\Theta(D)$ from one another, and every node in the graph is within some maximum distance $\Theta(D)$ from the nearest member. Computation of this ruling set by the agents corresponds, again, to a computation in the early-stopping LOCAL model.

[2]For instance, with $\ell$ the smallest of the two agents' labels, $D$ their initial distance, and simultaneous startup, the complexity of rendezvous is $\Theta(D \log \ell)$ in the anonymous ring [DFKP06].

To achieve rendezvous, each agent effectively substitutes its starting position with the nearest node on the sub-line. By having the agents agree on a common coloring of this sub-line, as before, the agents can now find each other by performing some waiting and searching phases based on the colors of their location in the sub-line. Even if our new sub-line is less structured than the one considered in the setting with knowledge of $D$, its properties are sufficient to be able to give distinct colors to members of the sub-line at distance $O(D)$ in the original line, while only using $O(1)$ colors.

Computing the ruling set we need in the early-stopping LOCAL model follows relatively straightforwardly from standard results in distributed computing. We perform this computation by repeated applications of an algorithm for Maximal Independent Set (MIS). At a high level, we start from a set containing all nodes of the lines and gradually remove nodes from it. Each call to the MIS subroutine roughly doubles the minimum distance between members of the set, while maintaining that the maximum distance of any node on the graph to the nearest member of the set increases only moderately. A small subtlety is that we have each node mostly coordinate with other nodes whose labels are of the same order of magnitude.

**A simplifying remark: avoiding simultaneous edge crossings.** Contributing to the overall simplicity of our algorithm is a subroutine for crossing edges that prevents the agents from exchanging positions on the line without achieving rendezvous. The idea behind it is very simple: on the labeled line, every edge can be given a canonical orientation, for example, from lower to higher ID. By having the agents perform a different set of moves to cross an edge depending on the direction in which they take it, we can make sure that the agents must be at the same node at some point during the crossing. The first few moves are the same for both orientations of the edge, which enables an agent to discover the orientation of the edge while crossing it – e.g., on the labeled line, through learning the identifier of the other endpoint. Replacing every one-round edge crossing by this procedure only incurs a multiplicative factor of 4 in the complexity of the algorithm. We believe that this observation might be useful to the design of future rendezvous algorithms on graphs whose edges can easily be given an orientation.

### 1.3 Related Work

Most relevant to the results of this paper is the work by Miller and Pelc [MP25], that considered the rendezvous problem with the exact same set of assumptions as we do in this paper, as well as some easier settings. They showed that when agents know the distance $D$ between themselves, and the maximum labels of the two starting nodes of the agents is $\ell$, deterministic rendezvous can be done in $O(D \log^* \ell)$ rounds. They gave a matching lower bound of $\Omega(D \log^* \ell)$, shown on a line whose IDs all have the same $\log^* \ell$. The lower bound holds even with simultaneous startup. The upper bound does not assume this. When $D$ is unknown, they give an upper bound of $O(D^2(\log^* \ell)^3)$. They also gave an algorithm to achieve rendezvous in $O(D)$ rounds when the agents know their positions on the infinite line, improving upon an earlier result by Collins et al. [CCG+11] which additionally assumed simultaneous startup of the agents.

On an anonymous line, where the agents themselves are given two distinct labels, the problem is known to be harder. Dessmark et al. [DFKP06] showed that the problem has complexity $\Theta(D \log \ell)$, assuming simultaneous startup, where $\ell$ is the smallest of the two agents' labels. With arbitrary startup, they consider a harder setting where the agents only appear in the graph once they are awake. With that harder assumption, the algorithms tend to have a dependency on $n$, the size if the graph (which is then not assumed to be infinite in this work). Rendezvous in infinite lines was first studied in [MGK+06], in which the authors considered the harder asynchronous setting, but with the agents capable of detecting each other in edges.

Before they were formulated as questions of computational complexity and algorithm design, coordination problems of agents trying to meet appeared in the economics literature [Sch81]. Algorithms for rendezvous in graphs were first studied by Alpern [Alp95]. Some of the graph

| Assumptions | | | | | | | | Bound | Citation |
|---|---|---|---|---|---|---|---|---|---|
| Lin | $n \ll \infty$ | NL | AL | EwA | SWU | DbA | LoG | | |
| ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | $\Omega(D)$ | folklore |
| ✓ | ✓ | | ✓ | | ✓ | | | $\Theta(D \log \ell)$ | [DFKP06] |
| ✓ | | (LoG) | (LoG) | ✓ | ✓ | | ✓ | $O(D)$ | [CCG+11] |
| ✓ | | (LoG) | (LoG) | ✓ | | | ✓ | $O(D)$ | [MP25] |
| ✓ | | ✓ | (NL) | ✓ | ✓ | ✓ | | $\Omega(D \log^* \ell)$ | [MP25] |
| ✓ | | ✓ | (NL) | ✓ | | ✓ | | $\Theta(D \log^* \ell)$ | [MP25] |
| ✓ | | ✓ | (NL) | ✓ | | | | $O(D^2(\log^* \ell)^3)$ | [MP25] |
| ✓ | | ✓ | (NL) | ✓ | | | | $\Theta(D \log^* \ell)$ | **this paper** |

Acronyms for assumptions: Lin (Graph is Line-like): the graph has maximum degree 2. $n \ll \infty$ (Finite Graph): when unchecked, the graph can be infinite. NL (Node Labels): each node is equipped with a unique label. Implied by LoG, and implies AL. AL (Agent Labels): the agents are each given a unique label. Implied by LoG and NL. EwA (Exists while Asleep): rendezvous is also achieved if an awake agent finds an asleep agent. SWU (Simultaneous Wake-Up): the agents wake up in the same round ($\tau = 0$). DbA (Distance between Agents): the agents know the initial distance $D$ between their starting locations. LoG (Location on Graph): the graph has some global naming scheme, which is known to the agents, and in which they know where they are. Implies NL and AL.

Table 1: Comparison of our result with some of the most relevant works in the literature.

structures in which deterministic rendezvous has been considered include the infinite tree [BP24], the infinite grid [GP25], the plane [DPP23] and the ring [KSSK03]. Deterministic rendezvous was also studied in general graphs [TZ14], and in a setting where agents are given additional information about their location in the graph [CCG+11]. Rendezvous on the line has also been studied with random resources [LA96, BG98].

The $\Omega(D \log^* \ell)$ lower bound obtained in [MP25] comes from a reduction of the rendezvous problem in labeled lines to the problem of coloring nodes on a line in the LOCAL model with some number of colors $C \geq 3$. This problem has been shown to have a $\Omega(\log^* n - \log^* C)$ lower bound [Lin92, LS14]. This lower bound is tight, due to an earlier algorithm by Cole and Vishkin matching its complexity [CV86]. Many works consider a version of the LOCAL model in which the nodes know an upper bound $n$ on the number of nodes, and have IDs between 1 and $\text{poly}(n)$. A generic way to remove this type of global knowledge was developed in a work by Korman, Sereni, and Viennot [KSV13]. Several ideas are transferable from the setting where $n$ is unknown to that of infinite graphs.

Readers interested in a broader view of the area of deterministic rendezvous might be interested in a recent survey by Pelc [Pel19]. The book by Alpern and Gal [AG03] offers an introduction to the area of rendezvous that includes many topics outside the scope of this paper (asymmetric agents, continuous versions of the problem). For an introduction to the LOCAL model, see the book by Peleg [Pel00], as well as the online book by Hirvonen and Suomela [HS20].

### 1.4 Organization of the Paper

We start with some preliminaries in Section 2, where we introduce the notations and the model we are going to be considering throughout the paper.

In Section 3, we present the techniques and algorithms that the agents are going to be using to break symmetry between them and achieve rendezvous. Section 3.1 shows how to devise a way for the agents to meet during the execution of the algorithm when they both move through

the same edge at the same time but from different starting nodes. In Section 3.2, we build an efficient algorithm to compute ruling sets on paths in LOCAL. Then, using this algorithm, we obtain in Section 3.3 an efficient algorithm for computing ruling sets on paths in LOCAL in which nodes with smaller IDs terminate earlier.

Section 4 describes an optimal algorithm for the agent to reach rendezvous in our setting and analyses its complexity. Section 4.1 presents the algorithm computed by the agents. To prove that the agents achieve rendezvous, we focus on a specific phase of the algorithm of one of the agents. We show that after reaching this phase the agents will achieve rendezvous before switching to the next phase. We split the analysis according to whether the agents are in the same phase or not. We argue about the two agents achieving rendezvous when in different phases in Section 4.3, and when in the same phase in Section 4.4. We additionally show that the phase of our algorithm for which our arguments hold is guaranteed to occur within our stated upper bound, and finish the proof of our optimal upper bound in Section 4.5.

We give in Section 5 the implications of our results for line-like graphs other than the infinite line: finite lines, finite cycles, and the infinite half-line (containing a single node of degree 1).

# 2 Preliminaries

## 2.1 Notation

Throughout the paper, log refers to the base-2 logarithm, i.e., $\log(2^x) = x$ for any $x \in \mathbb{R}$. For each $x \in \mathbb{R}$, the $\log^*$ function is defined as $\log^*(x) = 1$ if $x \leq 1$, $\log^*(x) = 1 + \log^*(\log(x))$. This is off-by-one from the maybe more common definition that sets $\log^*(x) = 0$ if $x \leq 1$. We do so to have that $\Theta(D \log^* \ell)$ is of order $\Theta(D)$ rather than 0 when $\ell = 1$.

**Graph notation.** We write $G = (V, E)$ for a graph over vertices $V$ and edges $E$. Each node $v \in V$ is equipped with a unique label or identifier $\ell(v) \in \mathcal{L}$. We identify the set of labels with the positive integers, $\mathcal{L} = \mathbb{N}^*$. We partition the nodes according to their labels and the $\log^*$ function, defining $V_i = \{v \in V : \log^*(\ell(v)) = i\}$, so that $V = \bigsqcup_{i \geq 1} V_i$. We also define $V_{\leq i} = \bigsqcup_{j=1}^{i} V_j$ and $V_{<i} = V_{\leq i} \setminus V_i$. The degree of a node $v \in V$ is denoted by $\deg(v)$, the distance $\mathrm{dist}(v, v')$ between two nodes corresponds to the (edge-)length of a shortest path between them, and $N(v)$ is the set of neighbors of $v$. For every integer $k$, $N^k(v) = \{u \in V : \mathrm{dist}(u, v) \in [1, k]\}$ is the $k$-hop neighborhood of $v$.

For $U \subseteq V$ a subset of the nodes of a graph $G = (V, E)$, the graph $G[U] = (U, E')$ is the graph induced by $U$, whose set of nodes is $U$ and whose set of edges is $E' = \{vv' \in E : v \in U \wedge v' \in U\}$. For a positive integer $k \geq 1$, the power graph of order $k$ is the graph $G^k = (V, E'')$ with the same set of nodes as $G$ and whose set of edges is $E'' = \{vv' : \mathrm{dist}_G(v, v') \in [1, k]\}$. Note that $G^k[U] = (G^k)[U]$ rather than $(G[U])^k$, i.e., $G^k[U]$ is an induced subgraph of a power graph of $G$, rather than a power graph of an induced subgraph of $G$.

**The Labeled Infinite Line.** An infinite labeled line is a 2-regular connected graph $G = (V, E)$ over a countably infinite set of nodes.

For the analysis' sake, it is sometimes convenient to consider a canonical orientation and naming for the line. When we do so, we refer to nodes on the infinite line as relative integers, s.t. for any $i \in \mathbb{Z}$, node $v_i$ is adjacent to nodes $v_{i-1}$ and $v_{i+1}$ on the line.

## 2.2 Computational Models

**Our Mobile Agent Model.** The agents are placed and move in a network modeled by a graph $G = (V, E)$. $G$ can be finite or countably infinite. Each node $v \in V$ of the graph has $\deg(v)$ ports, and each edge is the graph is matched to exactly one port of each of its two endpoints.

The ports of a node $v$ are labeled 0 to $\deg(v) - 1$. At a node $v$, an agent can read the node's label $\ell(v)$ if the graph is labeled, see the node's degree $\deg(v)$, and distinguish the $\deg(v)$ ports leading out of the node.

The agents execute the same deterministic algorithm, as in, when put in the same configuration, agents $\alpha$ and $\beta$ perform the same set of moves. Time is divided into discrete rounds. The agents are initially asleep, and upon wake-up (or startup), they have no notion of how much time has elapsed before. Formally, the agents have an internal clock that is initialized at 0 upon waking-up, and from there on keeps track of how many rounds passed since wake-up. We denote by $\tau$ the delay between the wake-up times of the two agents. The agents can be thought of as waking up in rounds 0 and $\tau$. The initial distance between the two agents is denoted by $D$.

In each round, an awake agent at a node $v$ chooses one of $\deg(v) + 1$ possible moves. One of these moves is staying at $v$ for one round, the other moves being to take one of the ports leading out of $v$. When an agent takes a port, in the next round, it arrives at the other endpoint of the edge behind that port. When entering a node, an agent knows the port it enters it from. We put no limit on the memory of the agents, and they can base each of their decisions on everything they have experienced in previous rounds: the labels of all the nodes they were at in previous rounds, and from which port they were all entered from.

The agents can detect each other if they are at the same node in the same round, in which case they achieve rendezvous. This is also the case if one of the agents is asleep, i.e., agents exist in the network before waking up. They have no other means of communication. By default, the agents do not detect each other if they cross the same edge in opposite directions in the same round. We consider agents able to detect simultaneous edge-crossing in some parts of the paper.

The agents have no shared notion of orientation of the line, no idea of their position on the line with respect to some shared point of origin, and have no knowledge of both $D$ and $\tau$.

**The LOCAL Model on Infinite Graphs.** The standard LOCAL model is a model of distributed computation in which a network is represented as a graph $G = (V, E)$ whose nodes possess computational power. Every node has a unique identifier, typically assumed to be an integer between 1 and $\text{poly}(n)$, where $n$ is the number of nodes in the graph. The time is measured in rounds, in which a node can receive the messages sent by its neighbors in the previous round, do some internal computation, and send a message to each of its neighbors for them to receive in the next round. Computation starts at the same time for all nodes and is done synchronously. An algorithm's complexity on instances of size $n$ is the maximum number of rounds before all nodes in the graph have terminated executing the algorithm and permanently committed to an output. The complexity of a problem on instances of size $n$ is the minimum complexity over all possible algorithms solving the problem over instances of size $n$. Messages sent in the LOCAL model can be arbitrarily large. As a result, without loss of generality, each node learns its entire $T$-hop neighborhood in $T$ rounds of LOCAL. This makes a $T$-round algorithm for a problem in the LOCAL model equivalent to a map from possible $T$-hop neighborhoods to possible node outputs.

In infinite graphs, it is nonsensical to consider the complexity of most problems as the time needed for the last node to terminate. The complexity also cannot be measured with respect to the number of nodes or the value of the largest identifier. To tame the infinite, we instead aim to bound the time it takes a node to finish with respect to some local parameters, for example, its identifier. To have nodes terminate at different times, the problems we consider must be inherently quite local. We refer to this model, where nodes commit to an output in a time that is a function of a local parameter, and thus algorithms make sense on both finite and infinite graphs, as the EarlyStop-LOCAL model (ES-LOCAL for short). This model was also used in the work by Miller and Pelc [MP25] and appeared previously in other sources [HS20, Exercise 1.7].

### 2.3 Useful Subroutines

Consider a graph $G = (V, E)$ and a subset $S \subseteq V$ of its nodes. The set $S$ is *independent* iff for all $u, v \in S$, $uv \notin E$. $S$ is *dominating* iff for all $u \in V \setminus S$, $\exists v \in S$, $uv \in E$. If $S$ is both independent and dominating, it is a *maximal independent set* (MIS).

Let each node $v \in V$ be given a list of colors $\psi(v)$. A list-coloring of the graph for these lists is a map $\chi : V \to \bigcup_{v \in V} \psi(v)$ s.t. for each node $v$, $\chi(v) \in \psi(v)$, and for any two adjacent nodes $u$ and $v$, $\chi(u) \neq \chi(v)$. When each node $v$ is given a list of size $|\psi(v)| \geq \deg(v) + 1$, the coloring is known as a *degree+1-list coloring*. Having lists with this minimum size guarantees the existence of a solution $\chi$. In fact, it also guarantees that a partial solution can always be extended to a full one. When the lists are all equal to $[k]$ for some integer $k$, the problem is known as $k$-coloring. In the extensively studied $\Delta + 1$-coloring problem, each node has $[\Delta + 1]$ as its list of colors, where $\Delta$ is an upper bound on the maximum degree of the graph.

Recall that in the LOCAL model, $n$ corresponds to the number of nodes in the graph, and $\text{poly}(n)$ bounds the nodes' identifiers.

**Lemma 2.1** ([Lin92])**.** *Both computing a Maximal Independent set and computing a degree+1-list coloring can be done in $O(\log^* n)$ rounds of LOCAL on $n$-node graphs of constant maximum degree $\Delta \in O(1)$.*

## 3 Our Main Methods

### 3.1 Ensuring Rendezvous from Crossing: CarefulWalk

Our model assumes that the agents have no detection mechanism other than that they know if they are at the same node in a given round, in which case they achieve rendezvous. In particular, the agents cannot detect if they cross the same edge in opposite directions in the same round, and this does not count as achieving rendezvous. In this section, we show that having a labeled line makes this distinction irrelevant.

We give a simple procedure (Algorithm 1) to cross edges which ensures that the agents cannot exchange positions on the line without achieving rendezvous. To do so, CarefulWalk takes 4 rounds to cross each edge. Its basic idea is to have the agents behave differently depending on whether they cross an edge in the direction of increasing or decreasing IDs. The same can be achieved given a common orientation of the edges instead of node labels.

**Algorithm 1** CarefulWalk, for crossing an edge $e = uv$

Let $u$ be the initial node, $v$ the target node, $e = uv$ the edge to cross.

1: Wait at initial node for one round. ▷ Agent at $u$ at $t = 0, 1$.
2: Cross the edge: $u \to v$. ▷ Agent at $u$ at $t = 1$, at $v$ at $t = 2$.
3: **if** $\text{ID}(v) > \text{ID}(u)$ **then**
4: Wait two rounds. ▷ Agent at $v$ at $t = 2, 3, 4$.
5: **else** ▷ $\text{ID}(v) < \text{ID}(u)$
6: Cross the edge: $v \to u$. ▷ Agent at $v$ at $t = 2$, at $u$ at $t = 3$.
7: Cross the edge: $u \to v$. ▷ Agent at $u$ at $t = 3$, at $v$ at $t = 4$.

Consider an algorithm $\mathcal{A}$ for the moves of an agent. Let care($\mathcal{A}$) be the algorithm obtained from $\mathcal{A}$ by replacing each edge-crossing in $\mathcal{A}$ by a call to CarefulWalk, and stretching each stay at a node by a factor of 4. This new algorithm basically does the same moves as $\mathcal{A}$ at a slower pace. We show in Theorem 3.1 that this transformed algorithm is able to achieve rendezvous as $\mathcal{A}$ would in the stronger setting where the agents can detect simultaneous edge-crossing.

**Theorem 3.1.** *Let $G$ be a labeled graph, with agents placed at two locations $v_\alpha$ and $v_\beta$ of the graph, $T$ an integer, and let $\mathcal{A}$ be a deterministic algorithm for the agents' moves. Suppose that*

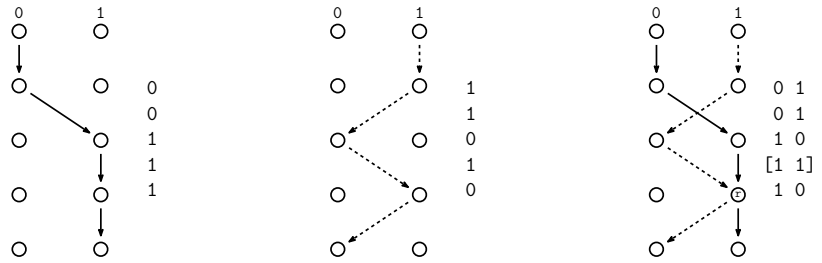



Figure 1: Illustration of the moves performed by CarefulWalk to cross an edge $uv$. The five circles below each 0 (resp. 1) represent the node of lower ID (resp. higher ID) at 5 successive time steps. Arrows represent the moves (or absence thereof) of an agent in a CarefulWalk crossing the edge. The left and middle figures depict crossing the edge in opposite directions. The right figure depicts two agents performing a CarefulWalk over the same edge simultaneously in opposite directions, achieving rendezvous at time step 4, after 3 moves. The 5-bit strings in the figures describe the positions occupied the agents over the 5 time steps.

*for any delay $\tau \geq 0$ between the wake-up times of the two agents, with simultaneous edge-crossing counting as rendezvous, the two agents achieve rendezvous in at most $T$ rounds when executing $\mathcal{A}$ on the graph $G$ with $v_\alpha$ and $v_\beta$ as the starting nodes for the two agents.*

*Let* care($\mathcal{A}$) *be the algorithm obtained from $\mathcal{A}$ by replacing every edge-crossing by a* CarefulWalk*, and every action of staying at a node by 4 such actions. Then in the setting without simultaneous edge-crossing, on the same graph $G$ and with the same starting nodes $v_\alpha$ and $v_\beta$,* care($\mathcal{A}$) *has the agents achieve rendezvous in at most $4T$ rounds, for any delay $\tau \geq 0$ between the agents' wake-up times.*

*Proof.* Let agent $\alpha$ be the first agent to wake up (in round 0), and $\beta$ the other agent (waking up in round $\tau \geq 0$). Consider the execution of care($\mathcal{A}$). In that execution, for each integer $i \geq 0$, let $x(i)$ be the position of agent $\alpha$ in round $i$, and $y(i)$ that of agent $\beta$. In particular, $y(i) = y(0)$ for all $i \leq \tau$.

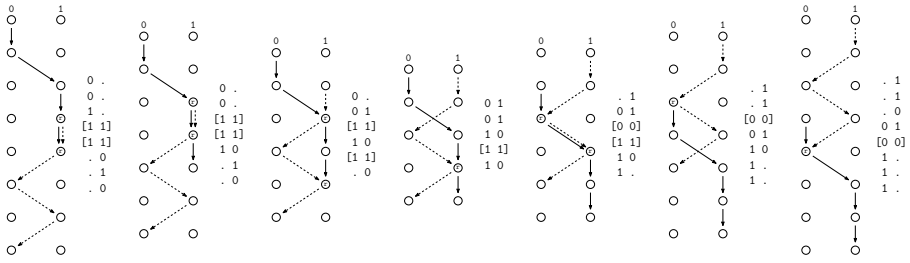
Let $\tau' = \lceil \tau/4 \rceil$ and $r = 4\tau' - \tau$. Let $x'(0) = x(0)$, $y'(0) = y(0)$, and for all $i \geq 1$, let $x'(i) = x(4i)$ and $y'(i) = y(4i - r)$. Note that the rounds $\{4i : i \in \mathbb{N}\}$ (resp. $\{4i - r : i \geq \tau'\}$) are exactly the rounds in which agent $\alpha$ (resp. $\beta$) finishes and starts a 4-round action in care($\mathcal{A}$) – staying put for 4 rounds, or doing a CarefulWalk. As a result, $x'(i)$ and $y'(i)$ are the positions that agents $\alpha$ and $\beta$ would have in round $i$ of an execution of the original algorithm $\mathcal{A}$ with a delay of $\tau'$.

If agents start at the same node, rendezvous is trivially achieved immediately. Otherwise, since $\mathcal{A}$ is guaranteed to achieve rendezvous in $T$ or less rounds (with simultaneous edge-crossings counting as rendezvous), there must exist some $t \in [1, T]$ such that either $x'(t) = y'(t)$, or $x'(t) = y'(t-1)$ and $y'(t) = x'(t-1)$. Let $t^* \geq 1$ be the smallest such value.

Suppose that $x'(t^*) = y'(t^*)$. If $r = 0$ (i.e., $\tau = 0 \mod 4$), then $\alpha$ and $\beta$ are at the same node $x(4t^*) = y(4t^*)$ in round $4t^*$. Otherwise ($r \in \{1, 2, 3\}$), consider the positions of the agents in rounds $4t^*$ and $4t^* - 2$. Regardless of whether $\alpha$ was staying idle for 4 rounds or performing a CarefulWalk to node $x(4t^*)$ in the 4 rounds prior to round $4t^*$, it was at node $x(4t^*)$ in rounds $4t^*$ and $4t^* - 2$ (see Algorithm 1 and Fig. 1, the description and an illustration of CarefulWalk). Similarly, regardless of the action that agent $\beta$ initiates in round $4t^* - r$, it is at node $y(4t^* - r) = x(4t^*)$ in rounds $(4t^* - r)$ and $(4t^* - r + 1)$. For all possible values of $r$, the agents meet in either round $4t^*$ or $4t^* - 2$.

Suppose that $x'(t^*) = y'(t^* - 1)$ and $x'(t^* - 1) = y'(t^*)$. A similar case analysis as the one we just did shows that the agents necessarily meet in a round in the interval $[4t^* - 4, 4t^* - 1]$ (see Fig. 2).

In all cases, agents executing care($\mathcal{A}$) achieve rendezvous in round $4T$ at the latest, proving the claim. □



(a) (b) (c) (d) (e) (f) (g)

Figure 2: The agents always meet when crossing the same edge in opposite directions using CarefulWalk.

Theorem 3.1 allows us to design algorithms for the simpler setting where agents detect simultaneous edge-crossing, and automatically translate rendezvous results about these algorithms to the setting where agents only achieve rendezvous if at the same node in a given round, which is the original setting of interest. This dramatically simplifies the work of the algorithm designer. Importantly, on the line, having a detection mechanism for simultaneous edge-crossing prevents the agents to swap positions without achieving rendezvous.

**Proposition 3.2.** *Consider a canonical naming of the nodes on the infinite line as relative integers, s.t. for any $i \in \mathbb{Z}$, node $v_i$ is adjacent to nodes $v_{i-1}$ and $v_{i+1}$ on the line. Let $a, b, c, d \in \mathbb{Z}$ be integers and $T, T'$ be two rounds with $T' > T$. For each $t \in [T, T']$, let $x(t) \in \mathbb{Z}$ (resp. $y(t) \in \mathbb{Z}$) be the location of agent $\alpha$ (resp. $\beta$) in round $t$. Consider the following two scenarios:*

1. *(Agents swap positions on the line) $a \leq b < c \leq d$, $x(T) = a < c = x(T')$ and $y(T) = d > b = y(T')$.*
2. *(An agent scans an interval that the other agent stays in) $a \leq b \leq c \leq d$, $\forall z \in [a, d]$, $\exists t \in [T, T']$ s.t. $x(t) = z$, and $\forall t \in [T, T']$, $y(t) \in [b, c]$.*

*If simultaneous edge-crossing counts as rendezvous, then in both scenarios, the agents necessarily meet in a round between $T$ and $T'$.*

### 3.2 Computing Ruling Sets on Paths

In this section, we give a LOCAL algorithm for computing what is essentially a ruling set over a path, but focusing on a subset of the nodes $U \subseteq V$.

**Definition 3.3** (Set-Limited Ruling Sets)**.** Let $\alpha$ and $\beta$ be two nonnegative integers, $G = (V, E)$ be a graph, and $U \subseteq V$ a subset of its nodes. A set of nodes $S$ is a $U$-limited $(\alpha, \beta)$-ruling set of $G$ if and only if:

1. $S \subseteq U$,
2. For any two nodes $u, v \in S$, $\text{dist}(u, v) \geq \alpha$ (packing property),
3. For any node $u \in U$, $\exists v \in S$, $\text{dist}(u, v) \leq \beta$ (covering property).

A standard ruling set of a graph $G = (V, E)$ is a $V$-limited ruling set of $G$. As simple examples of standard ruling sets, the whole set of nodes $V$ is an $(1, 0)$-ruling set, a maximal independent set of a graph $G$ is a $(2, 1)$-ruling set of that graph, and a maximal independent set of its order-$k$ power graph $G^k$ is a $(k + 1, k)$-ruling set of $G$.

Our $U$-limited ruling sets are exactly like standard ruling sets, except that they only need to cover nodes in $U$ and can only contain nodes from $U$. Importantly, they are not equivalent to

computing a standard ruling set of $G[U]$: in our definition of $U$-limited ruling sets, we measure the distances in the original graph $G$, not $G[U]$. Taking a finite set $U$ on an infinite graph $G$ means that its complexity can be measured as the longest time taken by a node to terminate, as in the standard LOCAL model, even as the communication takes place on an infinite graph. This algorithm will be the basis of our subsequent algorithm that performs a similar task, but additionally colors the ruling set and is early stopping.

**Algorithm 2** PathRulingSet.

**Input:** A finite set of nodes $U \subseteq V$ with IDs in $[n]$, an integer distance $R \geq 1$.
Let $d = \lceil \log R \rceil$, $S_0 \leftarrow U$. ▷ The ruling set is initially the entire set $U$
1: **for** $i = 1$ to $d$ **do** ▷ Each loop iteration computes $S_i \subseteq S_{i-1}$
2: **if** $i \leq d-1$ **then** Let $R_i \leftarrow 2^i - 1$ **else** Let $R_i \leftarrow R - 1$.
3: Consider the graph $H_i \leftarrow G^{R_i}[S_{i-1}]$.
4: Compute an MIS over $H_i$, let $S_i$ be its nodes.
5: Initialize $S_{\text{out}} \leftarrow S_d$.
6: **for** 6 iterations **do**
7: For each $v \in U$, compute $b_v = \min_{u \in S_{\text{out}}} \text{dist}(v, u)$. Let $U^{\geq R} = \{v \in U : b_v \geq R\}$.
8: Add to $S_{\text{out}}$ all $v \in U^{\geq R}$ s.t. $\forall u \in N^{R-1}(v) \cap U^{\geq R}, (b_u < b_v \vee (b_u = b_v \wedge \ell(v) > \ell(u)))$.
**Return:** $S_{\text{out}}$.

**Lemma 3.4.** *Executed by a set of nodes $U$ with identifiers between 1 and $n$ on an infinite labeled line, Algorithm 2 computes a $U$-limited $(R, R-1)$ ruling set of $G$ in $O(R \log^* n)$ rounds of deterministic LOCAL.*

*Proof.* $d = \lceil \log R \rceil$ implies that $2^d \geq R > 2^{d-1}$. We show that the following properties always hold for each integer $i \in [0, d-1]$: the nodes in the set $S_i$ are at distance at least $2^i$ from each other (packing property), and every node from $U$ is at distance at most $2^{i+1} - i - 2$ from a node in $S_i$ (covering property). We then analyze the distances for the last two sets $S_d$ and $S_{\text{out}}$. Finally, we use those properties to guarantee the $O(R \log^* n)$ runtime.

**Packing property.** For $i = 0$, the property trivially holds: $2^i = 1$ and two distinct nodes are necessarily at distance at least 1 from each other. For $i \in [1, d-1]$, the property follows directly from the fact that in iteration $i$ of the loop, we compute a maximal independent set over $H_i = G^{R_i}[S_{i-1}]$. Since the nodes in $S_i$ form an independent set in $G^{R_i}$, any two nodes in $S_i$ are not adjacent in $G^{R_i}$, thus are at distance at least $R_i + 1 = 2^i$ from each other in $G$, giving the claim. Additionally, nodes in $S_d$ are at least $R_d + 1 = R$-apart by the same argument. Finally, the nodes that are added to $S_d$ to obtain $S_{\text{out}}$ are selected to be at distance $\geq R$ from other nodes already in $S_{\text{out}}$ (condition $b_v \geq R$ in computing $U^{\geq R}$), and so that two nodes at distance $< R$ are never simultaneously added to $S_{\text{out}}$ (condition that $b_v$ is maximal among $U^{\geq R}$ nodes within a $(R-1)$-radius, using IDs as tie-breakers)

**Covering property.** Since $S_0 = U$, for $i = 0$, we have that every node in $U$ is at distance at most $0 = 2^{i+1} - i - 2$ from a node in $S_i$.

Consider an integer $i \in [1, d-1]$ and a node $v \in U$. Let $u \in S_{i-1}$ be the node that is nearest to $v$ in the set $S_{i-1}$. If this node is still in $S_i$, then $\text{dist}(v, S_i) = \text{dist}(v, S_{i-1})$. If not, then $u$ has a neighbor $u' \in G^{R_i}[S_{i-1}]$ that is still in the next independent set $S_i$, meaning that $\text{dist}(u, u') \leq R_i$. Thus, for any node $v \in U$, $\text{dist}(v, S_i) \leq \text{dist}(v, S_{i-1}) + R_i$.

Therefore for every integer $i \in [1, d-1]$, the maximum distance of a node in $U$ to a node in $S_i$ is at most $\sum_{j=1}^{i} R_j = \sum_{j=1}^{i} (2^j - 1) = 2^{i+1} - i - 2 \leq 2^d - d - 1 \leq 2R - 2$. The MIS computation over $S_d$ adds at most $R - 1$ to this distance, giving a maximum distance of $3R - 3$.

Finally, consider the computation of $S_{\text{out}}$, initialized as $S_d$. Consider an iteration of the second loop, and take the current values of $S_{\text{out}}$ and $(b_v)_{v\in U}$. Consider a node $v \in U$ s.t. $b_v \geq R$. A node in its distance-$(3R-4)$ neighborhood must be added to $S_{\text{out}}$. Indeed, we have $b_u \leq 3R-3$ for all nodes $u \in U$ from the established properties of $S_d$. This implies that within distance $2R-3$ from $v$, there is a node $w$ s.t. $b_w \geq R$ and $b_w \geq b_u$ for all $u \in N^{R-1}(w)$. Either this node is added to $S_{\text{out}}$, or a node $u \in N^{R-1}(w)$ of equal value $b_u = b_w$ is added to $S_{\text{out}}$.

Suppose that after $k$ iterations of the loop, for some node $v \in U$, it holds that $b_v \geq R$. This means that we had $b_v \geq R$ for the $k$ iterations of the loop, and therefore, $k$ nodes were added to $S_{\text{out}}$ in the distance-$(3R-4)$ neighborhood of $v$. A subsequent iteration number $k+1$ should add another node to $S_{\text{out}}$ within this neighborhood. But nodes in $S_{\text{out}}$ are at distance at least $R$ from one another, so the distance-$(3R-4)$ neighborhood of $v$ can contain at most 6 nodes in $S_{\text{out}}$. Therefore, after 6 iterations of the loop, all nodes $v \in U$ satisfy $b_v \leq R-1$, i.e., are at distance $< R$ from $S_{\text{out}}$.

**Complexity.** $G$ being a path/line, the fact that two nodes in $S_{i-1}$ are at distance at least $2^{i-1} \geq (R_i+1)/2$ from one another in $G$ bounds the maximum degree of $H_i = G^{R_i}[S_{i-1}]$ to 2. The nodes of these graphs are also all from $U$, which bounds their identifiers to be at most $n$. These two bounds imply that all the computations we do on each of these graphs can be done in $O(\log^* n)$ rounds of LOCAL on the said graph. Simulating a round of LOCAL on a graph $H \subseteq G^k$ can be done in $k$ rounds of communication over $G$. As we consider $\Theta(\log R)$ graphs whose costs of simulation increase geometrically, we get a total complexity of $O(\sum_{i=1}^{d} R_i \log^* n) = O(R \log^* n)$ for the $d$ MIS computations. The greedy extension of $S_d$ to $S_{\text{out}}$ only adds $O(R)$ rounds to this complexity, as $O(R)$ rounds suffice for each node to compute its distance $\leq 3R-3$ to the nearest node in $S_d$ and compare it to the values of the nodes within distance $R-1$. □

### 3.3 Computing Colored Ruling Sets on Paths, and Stopping Early

The previous section considered the standard LOCAL model of distributed computing, applied to a finite set of nodes (at most $n$, due to the bound on the nodes' identifiers), even if on an infinite line. In this section, we use this algorithm to compute the same kind of object on the infinite line, in the Early Stop paradigm.

Our algorithm gradually computes a ruling set $S$ of the infinite line, in the sense that we provide an algorithm in which every node on the infinite line eventually decides whether it is part of the ruling set or not, and in any given round, the set of nodes that committed to their output and joined the set $S$ is a ruling set over the nodes that committed to their output. At a high level, Algorithm 3 partitions the nodes on the infinite line according to their IDs, and computes a ruling set over each set of nodes of this partition using Algorithm 2 (PathRulingSet) from the previous section. The computation done by the set of nodes $V_i = \{v \in V : \log^* \ell(v) = i\}$ is well defined as it is performed by a finite set of nodes, with bounded IDs. After $\Theta(R \cdot i)$ rounds, the nodes in $V_i$ have all finished their execution of PathRulingSet, as did previously nodes in the sets $V_j$, $j < i$. Once $V_i$ has computed a $V_i$-limited ruling set $S_i$, we use it to extend the current ruling set $S$, that only contains nodes from $V_{<i}$ at this point. This is done greedily. We first have each node $v \in S_i$ join $S$ if and only if $S$ does not already contain a node at distance $< R$ from $v$ on the line. A second greedy extension slightly improves the covering distance. Finally, we assign a color to the new members of the ruling set.

**Lemma 3.5.** *For any round $T$, let $U_T$ be the set of nodes that committed to their outputs by round $T$, $S_T \subseteq U_T$ the set of nodes that joined $S$ by round $T$, and $\chi_T : S_T \to [17]$ the colors assigned to nodes in $S_T$. Algorithm 3 achieves the following:*

1. *A node $v$ of label $\ell(v)$ terminates in $O(R \log^* \ell(v))$ rounds,*

**Algorithm 3** ES-ColPathRulingSet.

**Input:** An infinite labeled line $G$, an integer distance $R \geq 1$.

$S \leftarrow \emptyset$ current ruling set, $\chi : S \to [17]$ current coloring.
**for each** $i \in \mathbb{N}^*$ in parallel **do**
  Compute a $V_i$-limited $(R, R-1)$-ruling set with PathRulingSet.
  Let $S_i$ be the set and $\chi_i$ be the coloring of $S_i$ computed on the previous line.
  Wait for some $\Theta(R \cdot i)$ rounds, ensuring that nodes in $V_{<i}$ all chose their output.
  Let $S_{<i}$ be the value of $S$ at this stage. ▷ $S_{<i}$ is a $V_{<i}$-limited $(R, R-1)$-ruling set
  Let $S'_i \leftarrow S_i \setminus N^{R-1}(S)$.
  Update $S \leftarrow S \cup S'_i$.
  **for** 4 iterations **do**
    For each $v \in V_i$, compute $b_v = \min_{u \in S} \mathrm{dist}(v, u)$. Let $U_i^{\geq R} = \{v \in V_i : b_v \geq R\}$.
    Add to $S$ all $v \in U_i^{\geq R}$ s.t. $\forall u \in N^{R-1}(v) \cap U_i^{\geq R}, (b_u < b_v \vee (b_u = b_v \wedge \ell(v) > \ell(u)))$.
  Each $u \in S \setminus S_{<i}$ computes $\psi(u) = [17] \setminus \{\chi(w) : w \in S_{<i} \cap N^{9R-1}(u)\})$.
  Compute a list-coloring of $G^{9R-1}[S \setminus S_{<i}]$ according to the lists $(\psi(u))_{u \in S \setminus S_{<i}}$.
  Extend $\chi$ from $S_{<i}$ to $S$ with this coloring.
  Each node $v \in V_i$ learns $N^{R-1}(v) \cap S$ and $\chi(u)$ for each $u \in N^{R-1}(v) \cap S$.
  Nodes in $V_i$ commit to their output.
**end**

2. *At termination, a node $v$ outputs whether it is part of the ruling set $S$ or not, and if part of $S$, it also outputs a color between 1 and 17. If outside $S$, $v$ knows the members of $S$ at distance $\leq R-1$ from itself together with their colors.*

3. *$S_T$ is a $U_T$-limited $(R, R-1)$-ruling set, and $\chi_T$ is a proper coloring of $G^{9R-1}[S_T]$.*

*Proof.* We first show that Algorithm 3 (ES-ColPathRulingSet) constructs the claimed object, before bounding its complexity.

**Properties of the Ruling Set.** Recall that $S_i$, the output of PathRulingSet executed by nodes in $V_i$, is a $V_i$-limited $(R, R-1)$-ruling set. Consider the set $S_{<i}$, the content of $S$ before adding nodes from $V_i$ to it. Assume $S_{<i}$ to be a $V_{<i}$-limited $(R, R-1)$-ruling set. We show that the nodes from $V_i$ added to it make $S$ a $V_{\leq i}$-limited $(R, R-1)$-ruling set. By induction, starting from the trivial base case of $S_{<1}$, we get the claimed properties about our ruling set.

Every node $v \in V_i$ is at distance at most $R-1$ from a node $u \in S_i \subseteq V_i$. Consider the state of $S$ after adding $S'_i = S_i \setminus N^{R-1}(S)$ to it. If $u$ was added to $S$, then $v$ remains at distance at most $R-1$ from a node in $S$. If $u$ was not added to $S$, there exists a node $w \in S_{<i}$ that joined $S$ earlier, such that $\mathrm{dist}(u, w) < R$. This implies that $v$ is at distance at most $\mathrm{dist}(v, u) + \mathrm{dist}(u, w) \leq 2R-2$ from a node in $S$. At this state, $S = S_{<i} \cup S'_i$ is a $V_{\leq i}$-limited $(R, 2R-2)$-ruling set. The 4 iterations of the inner loop that add to $S$ some of the nodes in $V_i$ furthest away from the current set $S$ make $S$ a $V_{\leq i}$-limited $(R, R-1)$-ruling set, from similar arguments as used in the proof of Lemma 3.4.

Finally, the minimum distance of $R$ between nodes in $S$ ensures a maximum degree of 16 for the graph $G^{9R-1}[S]$. Each list $\psi(u)$ for $u \in S \setminus S_{<i}$ therefore contains at least $\deg(u) + 1$ colors, where $\deg(u)$ is the degree of $u$ in $G^{9R-1}[S]$. As the lists are chosen so as to avoid creating conflicts with previously colored nodes in $S$, the extended coloring $\xi$ is a valid coloring of $G^{9R-1}[S]$.

**Complexity.** Recall that PathRulingSet (Algorithm 2) with parameter distance $R \geq 1$ has a complexity of $O(R \log^* n)$ rounds when run by nodes of maximum ID $n$. Nodes in $V_i$ thus spend $O(R \cdot i)$ rounds computing $S_i$ at the beginning of the (parallel) loop in Algorithm 3.

For every $i \in \mathbb{N}^*$, let $T_i$ be the round in which nodes in $V_i$ terminate in Algorithm 3. Nodes in $V_i$ first compute $S_i$, before greedily adding nodes of $S_i$ to $S$. Before adding nodes of $S_i$ to $S$, all nodes in $V_j, j < i$ must have terminated, which requires waiting until round $T_{i-1}$ (0 for the first set of nodes). After this amount of time, inserting the nodes of $S_i$ into $S$ only takes $R-1$ rounds, for each node in $S_i$ to learn which nodes in its $(R-1)$-neighborhood have joined $S$. The 4 loop iterations in which additional nodes from $V_i$ are added to $S$ are also completed in $O(R)$. Finally, solving the list-coloring problem is done in $O(R \cdot i)$ rounds (Lemma 2.1), and each node learning the output of nodes within distance $R$ takes $O(R)$ rounds.

In total, we have that $T_i \leq \max(O(R \cdot i), T_{i-1}) + O(R \cdot i)$. By induction, we get that $T_i \in O(R \cdot i)$. □

# 4 Our Optimal Rendezvous Algorithm

## 4.1 The Algorithm

In this section, we describe our deterministic rendezvous algorithm. We actually describe an algorithm working in the setting where agents achieve rendezvous when crossing the same edge in opposite directions. By Theorem 3.1, this implies an algorithm with similar asymptotic complexity for the setting where agents only detect each other if at the same node in a given round.

Recall that both agents behave according to the same algorithm, which is just a sequence of moves. In most of this section, we take the perspective of one of the two agents, which we will denote as agent $\alpha$. We call agent $\beta$ the other agent. $v_\alpha$ and $v_\beta$ are as before the starting nodes of agents $\alpha$ and $\beta$.

The two basic building blocks of our algorithm are ZWalk and ES-ColPathRulingSet, respectively Algorithm 4 and Algorithm 3. ZWalk is simply a procedure in which the agent executing it scans the $2L+1$ nodes of the distance-$L$ neighborhood of the node it starts the ZWalk from, in $4L$ rounds. Note that while the agents do not have a common orientation, they have a sense of direction in that they know the port from which they enter a node. This allows the agents to keep walking in the same direction for successive steps, and to change direction, as desired. Most moves taken during our algorithm are done in executions of ZWalk. ES-ColPathRulingSet is the early-stopping algorithm for computing colored ruling sets presented in the previous section.

**Algorithm 4** ZWalk($L$), agent $\alpha$ moves on the path up to distance $L$ from $v$ on both sides

1: Agent $\alpha$ takes $L$ steps in one direction.
2: Agent $\alpha$ takes $2L$ steps in the opposite direction.
3: Agent $\alpha$ takes $L$ steps in the first direction, getting back to its original position.

Our algorithm (Algorithm 5) consists of a loop with a parameter $L$ that doubles between iterations. Inside the loop are two phases, which we call the *discovery phase* and the *searching phase*. In the discovery phase, an agent explores the distance-$L$ neighborhood around their starting position, roughly doubling how much of the network they know about. The agents use this information to compute a colored ruling set of parameter $R$, where $R \leq L/16$ is the largest power of 4 s.t. the distance-$O(R)$ neighborhood of their starting location contains a ruling-set node. Of course, as the agents have different views, they do not necessarily end up computing a ruling-set with the same parameter $R$, and we denote by $R_{\alpha,L}$ and $R_{\beta,L}$ the distance of the ruling sets computed by agents $\alpha$ and $\beta$ in their phase $L$. The agents use their respective computed ruling set to search for one another, using procedure SearchingWalk (Algorithm 6), which constitutes the searching phase.

In SearchingWalk (Algorithm 6), each agent goes to a nearby colored node of the computed

**Algorithm 5** Rendezvous algorithm for agent $\alpha$ starting at node $v_\alpha$

Let $\kappa_3 > 0$ be a universal constant s.t. any node $v$ executing ES-ColPathRulingSet($R$) computes its output in at most $\kappa_3 R \log^* \ell(v)$ rounds.

Initialize $L \leftarrow 1$.
**while** not(RendezVous) **do**
  ZWalk($L$). ▷ Discovery phase
  For each $R \in \{4^j \mid j \in \mathbb{N}\}$, let $S_R = \{u \in N^R(v_\alpha), N^{\kappa_3 R \log^*(u)}(u) \subseteq N^L(v_\alpha)\}$
  **if** $\exists R \in \{4^j \mid j \in \mathbb{N}\}$, $R \leq L/16$, s.t. $S_R \neq \emptyset$ **then**
    Let $R^* = \max\{R \in \{4^j \mid j \in \mathbb{N}\} : R \leq L/16 \wedge S_R \neq \emptyset\}$.
    Simulate ES-ColPathRulingSet($R^*$) for all nodes $u \in S_{R^*}$.
    Let $r$ be the known colored ruling set node closest to $v_\alpha$, breaking ties with IDs.
    SearchingWalk($R^*, L, r, \chi(r)$). ▷ Searching phase
  **else** ▷ Not enough nodes known to run ES-ColPathRulingSet
    Wait for $24L$ rounds.
  $L \leftarrow 2L$.

ruling set. Once at this node, the agents alternate calls to ZWalk and waiting periods, according to the color of the ruling set node. The fact that close enough ruling set nodes have distinct colors guarantees that the two agents, if they have computed a ruling set with the same distance $R \geq D$, will have different behaviors at some point. That is, one of the agents will be performing a ZWalk while the other is not moving. The agents achieve rendezvous as this happens.

**Algorithm 6** SearchingWalk($R, L, r, c$), in which agent $\alpha$ searches in the distance-$8R$ neighborhood of node $r$ close to its initial position $v_\alpha$

Move to node $r$. ▷ at distance $\leq 2R - 1$ from $v_\alpha$
Wait for $L - \text{dist}(v_\alpha, r)$ rounds. ▷ $\geq 0$ since $L \geq 16R$
ZWalk($8R$). ▷ $32R$ rounds
**for** $i$ from 0 to 4 **do** ▷ iterating over the bits of $c = \chi(r) \in \{0,1\}^5$
  **if** $c_i = 1$ **then**
    ZWalk($8R$) twice. ▷ $2 \times 32R$ rounds
  **else** ▷ $c_i = 0$
    Wait for $64R$ rounds.
Wait for $11 \cdot (2L - 32R)$ rounds. ▷ $\geq 0$ since $L \geq 16R$
Wait for $L - \text{dist}(v_\alpha, r)$ rounds.
Move back to node $v_\alpha$.

## 4.2 Analysis Overview

Missing from the previous section's rough overview is notably how the argument is robust to the fact that agents can have a delay of $\tau$ between their wake-up times. We now explain how, in the next sections, we analyze our algorithm and prove Theorem 1.1, that is, that the agents achieve rendezvous in $O(D \log^* \ell_{\min})$ rounds when executing Algorithm 5.

**Organization of the proof.** To prove Theorem 1.1, we first bound in Section 4.3 how out-of-sync the two agents can be without trivially achieving rendezvous. Knowing that a delay $\tau$ that exceeds some threshold $\Theta(D)$ necessarily implies that the agents achieve rendezvous in $O(D)$ rounds, the rest of the analysis can assume some degree of synchronization between the agents.

We then argue that as the agents reach a large enough value of $L$ (of order $\Theta(D \log^* \ell_{\min})$) in their executions of Algorithm 5, they compute nodes from colored ruling sets parameterized

by distances of order at least $\Omega(D)$. When using such ruling set nodes in their executions of SearchingWalk (Algorithm 6), the agents can achieve rendezvous in two different ways. If the two agents have computed nodes from distinct ruling sets (i.e., the distance parameters of their ruling sets do not match), the agent with the larger distance is guaranteed to find the other agent. This follows from the agent with the smaller distance parameter being constrained to some interval on the line, which we show gets fully explored by the agent with the larger parameter. We show this in Section 4.4.1. The analysis for when the two agents computed nodes from the same ruling set (i.e., with the same distance parameter) is in Section 4.4.2. There, we argue that rendezvous is easily achieved if the two agents decided to go to the same node of the ruling set, and if not, they find each other from the fact that they perform SearchingWalk from nodes that are both close enough to one another, and are colored with distinct colors. The various arguments are put together in Section 4.5.

**Notation and preliminary remarks.** We use the following notation:

- For each $L \in \{2^i \mid i \in \mathbb{N}\}$, $T_{\alpha,L}$ is the round in which agent $\alpha$ starts its discovery phase with parameter $L$ (line 3 in Algorithm 5). $T_{\beta,L}$ is the equivalent for agent $\beta$.
- $T_{\mathsf{rdv}} \in \mathbb{N} \cup \{\infty\}$ is the round in which the agents first achieve rendezvous ($\infty$ if the agents never do so).
- As previously in the paper, we identify the infinite line with the relative integers. For each $t \in \mathbb{N}$, let $x(t)$ (resp. $y(t)$) be the position of agent $\alpha$ (resp. $\beta$) in round $t$. We assume $\alpha$ to be the first agent to wake up, so that $y(t) = y(0) = v_\beta$ for all $t \leq \tau$.
- For each $L \in \{2^i \mid i \in \mathbb{N}\}$, let $R_{\alpha,L} \in (\{4^i \mid i \in \mathbb{N}\} \cap [1, L/16]) \cup \{\perp\}$ be the value of $R^*$ for agent $\alpha$ for the loop iteration of parameter $L$. $R_{\beta,L}$ represents the same quantity for $\beta$, and the value $\perp$ represents the fact that no valid $R^*$ was found for this loop iteration.

A useful remark about our algorithm is that the discovery and searching phases of our agents have fixed runtimes.

**Proposition 4.1.** *For each $L \in \{2^i \mid i \in \mathbb{N}\}$, we have:*

1. $T_{\beta,L} = T_{\alpha,L} + \tau$,
2. $T_{\alpha,L} = 28(L-1)$.

*Proof.* The loop iteration of Algorithm 5 with a given value of $L$ takes $28L$ rounds. Indeed, the discovery phase is a ZWalk of parameter $L$, taking $4L$ rounds. Whether some $R^* = 4^j$ is found such that $\exists u \in N^{R^*}(v_\alpha), N^{\kappa_3 R^* \log^*(u)}(u) \subseteq N^L(v_\alpha)$ or not, the rest of the loop always takes $24L$ rounds. Either these rounds are spent waiting at $v_\alpha$, or they are spent executing SearchingWalk (Algorithm 6). SearchingWalk has this complexity that is a function of $L$ only because each of its actions whose runtime increases with $R$ is compensated by a waiting period whose time decreases with $R$ by the exact same amount. $2 \times L$ rounds are spent traveling and waiting between $v_\alpha$ and $r$, and $11 \times 2L$ are spent in 11 calls to ZWalk($8R$) and/or waiting periods, for a total of $24L$ rounds.

The results follow from the just established formulas $T_{\alpha,2L} = T_{\alpha,L} + 28L$ and $T_{\beta,2L} = T_{\beta,L} + 28L$, together with the base case $T_{\beta,1} = T_{\alpha,1} + \tau$. □

## 4.3 First Argument: Out-of-Sync Rendezvous

In this section, we show that the agents achieve rendezvous relatively fast and easily when the delay $\tau$ is large enough. For a high-level intuition, assume that $\tau$ is very large compared to $D$. A property of our algorithm is that after $t$ rounds of being active, an agent has explored a

neighborhood of size $\Theta(t)$ around its starting location. With a distance of $D$ between the two agents' starting locations, this means that if after $\Theta(D)$ rounds of being active, agent $\alpha$ has reached the starting location of agent $\beta$. If $\beta$ is still asleep as this happens, we have that agent $\alpha$ finds it as it reaches $v_\beta$ for the first time. We essentially show an extension of this for smaller values of $\tau$.

We make some simple observations about the positions of the agents, from which it immediately follows that a delay of $10D$ or more makes the agents achieve rendezvous in $O(D)$ rounds.

**Proposition 4.2.** *Let $L = 2^i$ for some integer $i$.*

1. *For all $t \leq T_{\alpha,L} + L/2$, $|x(t) - v_\alpha| \leq L/2$.*
2. *For all $z \in [v_\alpha - L, v_\alpha + L]$, $\exists t \in [T_{\alpha,L}, T_{\alpha,L} + 3L]$ s.t. $x(t) = z$.*

*Proof.* For the first inequality, first note that by definition $x(T_{\alpha,L}) = x(0) = v_\alpha$ since the agent always returns at $v_\alpha$ before starting a new loop iteration. This immediately implies that $|x(t) - x(0)| \leq L/2$ for $t \in [T_{\alpha,L}, T_{\alpha,L} + L/2]$, as agents can only cross one edge per round. For smaller $t$ ($t \in [0, T_{\alpha,L}]$, let us consider the movements done in loop iterations for smaller values of $L$. As an agent executes a ZWalk with some parameter $d$, it stays at distance $\leq d$ from the node where it started its execution. As a results, all calls to ZWalk as part of previous discovery phases have kept the agent at distance $\leq L/2$ from $x(0) = v_\alpha$. As for the movements done in the searching phase, the agent moves at distance at most $2R^* - 1 + 8R^* < L$ from $v_\alpha$. Indeed, at most $2R^* - 1$ are done to go to a nearby ruling set node, from which ZWalk is called with parameter $8R^*$, and $R^* \leq L/16$.

The second item simply follows from the fact that a ZWalk of parameter $L$ consists of going $L$ steps away from the starting position, then taking $2L$ steps in the other direction. Therefore, either $x(T_{\alpha,L} + L) = v_\alpha - L$ and $x(T_{\alpha,L} + 3L) = v_\alpha + L$, or $x(T_{\alpha,L} + L) = v_\alpha + L$ and $x(T_{\alpha,L} + 3L) = v_\alpha - L$. In either case, the entire interval $[v_\alpha - L, v_\alpha + L]$ is swept. □

**Lemma 4.3.** *Let $L = 2^{\lceil \log D \rceil + 1}$. Either $\tau < 5L/2 \leq 10D$, or $T_{\mathsf{rdv}} \leq T_{\alpha,L} + 3L \in O(D)$.*

*Proof.* Note that $L \in [2D, 4D - 1]$. If $\tau \geq 5L/2$, then for all $t \leq T_{\alpha,L} + 3L \leq T_{\alpha,L} + \tau + L/2 = T_{\beta,L} + L/2$, $|y(t) - v_\beta| \leq L/2$ by Proposition 4.2. Because $|v_\alpha - v_\beta| = D$, this also implies that $|y(t) - v_\alpha| \leq D + L/2 \leq L$ for all $t \leq T_{\alpha,L} + 3L$. Since $\alpha$ explores the entire distance-$L$ neighborhood of $v_\alpha$ between $T_{\alpha,L} + L$ and $T_{\alpha,L} + 3L$ by Proposition 4.2, the two agents achieve rendezvous before round $T_{\alpha,L} + 3L$ by Proposition 3.2. □

### 4.4 Second Argument: In-Sync Rendezvous

Suppose now that the delay between the agents is less than $10D$. This implies that for sufficiently large values of $L$ (greater than some $\Theta(D)$), the discovery and searching phases of parameter $L$ of the two agents mostly overlap. We argue now that in this case, the agents can rely on the searching phase to achieve rendezvous, once $L$ has reached a large enough value (of order $\Theta(D \log^* \ell_{\min})$).

Recall that $R_{\alpha,L}$ and $R_{\beta,L}$ are the values $R^*$ computed by the two agents in the loop iteration of parameter $L$. $R_{\alpha,L}$ is the maximal value $R \in \{4^i \mid i \in \mathbb{N}\} \cap [0, L/16]$ such that for some node $u$ within distance $R$ from $v_\alpha$, the output of $u$ in ES-ColPathRulingSet($R$) can be inferred from nodes within distance $L$ from $v_\alpha$. $R_{\alpha,L} = \perp$ indicates that no such value $R$ exists for the loop iteration $L$. $R_{\beta,L}$ is the same variable for agent $\beta$.

The output of a node $u$ in ES-ColPathRulingSet($R$) includes a node from a $(R, R-1)$-ruling set within distance $R - 1$ from $u$ (possibly $u$ itself), by Lemma 3.5. In a loop iteration where $R_{\alpha,L} \neq \perp$, this means that $\alpha$ has learned about a node from the $(R_{\alpha,L}, R_{\alpha,L} - 1)$-ruling set computed by algorithm ES-ColPathRulingSet($R_{\alpha,L}$) at distance at most $2R_{\alpha,L} - 1$ from $v_\alpha$. We denote $r_{\alpha,L}$ this node, and $c_{\alpha,L}$ the color it got imparted in ES-ColPathRulingSet($R_{\alpha,L}$).

Recall that $\ell_{\min}$ is the smallest label within distance $D$ from one of the agents' starting locations, i.e., $\min\{\ell(u) \mid u \in N^D(v_\alpha) \cup N^D(v_\beta)\}$, and that $\kappa_3$ is a universal constant s.t. a node $u$ of label $\ell(u)$ executing ES-ColPathRulingSet($R$) computes its output in at most $\kappa_3 R \log^* \ell(u)$ rounds.

**Lemma 4.4.** *If $L \geq 8\kappa_3 D \log^* \ell_{\min} + 128D$ and $L \in \{2^i \mid i \in \mathbb{N}\}$, we have $R_{\alpha,L} \geq 2D$ and $R_{\beta,L} \geq 2D$.*

*Proof.* Let $v_{\min} \in N^D(v_\alpha) \cup N^D(v_\beta)$ be the node s.t. $\ell(v_{\min}) = \ell_{\min}$ and $R_D = 4^{\lceil \log_4(2D) \rceil}$. Note that $2D \leq R_D < 8D$, $\text{dist}(v_{\min}, v_\alpha) \leq 2D \leq R_D$, and $\text{dist}(v_{\min}, v_\beta) \leq 2D \leq R_D$.

Consider lines 5 and 6 in Algorithm 5. $R_{\alpha,L}$ is the largest value that satisfies the following three properties: $R_{\alpha,L}$ is a power of 4, $R_{\alpha,L} \leq L/16$, and agent $\alpha$ has discovered enough of the line in its discovery phase of parameter $L$ to compute the output of at least one node within distance $R_{\alpha,L}$ from $v_\alpha$ in an execution of ES-ColPathRulingSet($R_{\alpha,L}$). $R_{\beta,L}$ is defined similarly for agent $\beta$. We show that all these properties hold for $R_D = 4^{\lceil \log_4(2D) \rceil}$ with the lower bound on $L$ given as hypothesis, which implies the lemma.

$R_D$ is a power of 4 by definition. Also, since $L \geq 128D$ and $R_D < 8D$, we have $R_D \leq L/16$. Node $v_{\min}$ is at distance at most $2D \leq R_D$ from $v_\alpha$. Finally, consider the distance-$(\kappa_3 R_D \log^* \ell_{\min})$ neighborhood of $v_{\min}$. From the bound on the distance between $v_{\min}$ and $v_\alpha$, and $R_D < 8D$, it holds that $N^{\kappa_3 R_D \log^* \ell_{\min}}(v_{\min}) \subseteq N^{8\kappa_3 D \log^* \ell_{\min} + 2D}(v_\alpha) \subseteq N^L(v_\alpha)$. This implies that $S_{R_D} \neq \emptyset$ for $S_{R_D}$ defined as in lines 5 and 6 of Algorithm 5, and therefore, $R_{\alpha,L} \geq R_D \geq 2D$. The same holds for agent $\beta$. □

### 4.4.1 Mismatched Ruling Sets Rendezvous

We first consider the case where one of the agents has computed part of a ruling set of higher distance $R^*$ than the other agent. As $R_{\alpha,L}$ and $R_{\beta,L}$ are powers of 4, $R_{\alpha,L} > R_{\beta,L}$ implies that $R_{\alpha,L} \geq 4R_{\beta,L}$. This large difference between the two distances allows us to prove rendezvous by similar arguments as in the previous section (Lemma 4.3).

**Lemma 4.5.** *Suppose that $R_{\alpha,L} \neq R_{\beta,L}$ and $\max(R_{\alpha,L}, R_{\beta,L}) \geq D$. Then $T_{\mathsf{rdv}} \leq T_{\alpha,L} + 7L \in O(L)$.*

*Proof.* Assume $\tau \leq 10D$ as otherwise Lemma 4.3 already yields the claim.

Let agent $\alpha$ have computed the ruling set of higher distance, i.e., $R_{\alpha,L} > R_{\beta,L}$ and $R_{\alpha,L} \geq D$. Note that $R_{\alpha,L} \leq L/16$, so $L \geq 16D \geq 8\tau/5$. In the first $L$ rounds of SearchingWalk (rounds $T_{\alpha,L} + 4L$ to $T_{\alpha,L} + 5L$), $\alpha$ moves to the ruling set node $r_{\alpha,L}$ at distance at most $2R_{\alpha,L} - 1$ from $v_\alpha$. It then performs a ZWalk of parameter $8R_{\alpha,L}$. As a result, all the nodes within distance $8R_{\alpha,L} - (2R_{\alpha,L} - 1) - D > 5R_{\alpha,L}$ from $v_\beta$ are explored by $\alpha$ between round $T_{\alpha,L} + 5L$ and $T_{\alpha,L} + 6L$.

Consider agent $\beta$ during the same rounds. We know that $\beta$ started its discovery phase of parameter $L$ in round $T_{\beta,L} = T_{\alpha,L} + \tau \leq T_{\alpha,L} + L$. Therefore, between round $T_{\alpha,L} + 5L$ and $T_{\alpha,L} + 6L$, $\beta$ has finished its discovery phase of parameter $L$ and is either staying idle at $v_\beta$ or executing SearchingWalk. Either way, $\beta$ remains within distance $(2R_{\beta,L} - 1) + 8R_{\beta,L} \leq (5/2)R_{\alpha,L}$ from $v_\beta$ in those rounds.

Therefore, between round $T_{\alpha,L} + 5L$ and $T_{\alpha,L} + 6L$, $\beta$ stays within an interval of nodes while $\alpha$ fully explores that interval of nodes. By Proposition 3.2, the agents achieve rendezvous by round $T_{\alpha,L} + 6L$. The case $R_{\beta,L} > R_{\alpha,L}$ is similar, only introducing an extra $\tau \leq 10D \leq L$ to the number of rounds needed to achieve rendezvous. □

### 4.4.2 Common Ruling Set Rendezvous

The only case that remains to analyze is that where $R_{\alpha,L} = R_{\beta,L}$. We first show that if the agents go to the same node from the ruling set, they easily achieve rendezvous. Otherwise, they

achieve rendezvous as they perform successive calls to ZWalk according to the (distinct) colors given to $r_{\alpha,L}$ and $r_{\beta,L}$ by ES-ColPathRulingSet.

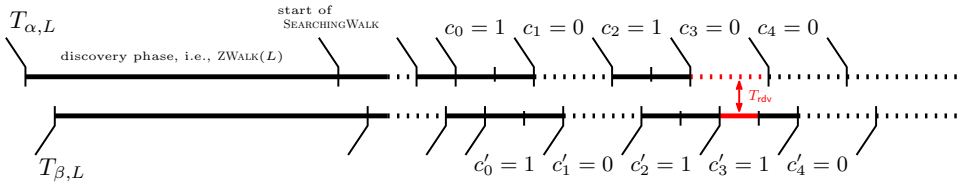



Figure 3: Timeline of an execution of SearchingWalk by the two agents, with colors $c = \texttt{10100}$ and $c' = \texttt{10110}$. Horizontal solid lines represent movement (mainly, calls to ZWalk) while dotted lines represent waiting. Rendezvous occurs during the processing of bit $i = 3$.

**Lemma 4.6.** *Suppose that $R_{\alpha,L} = R_{\beta,L} \geq D$ and $r_{\alpha,L} = r_{\beta,L}$. Then $T_{\mathsf{rdv}} \leq T_{\alpha,L} + 5L \in O(L)$.*

*Proof.* As before we assume that $\tau \leq 10D$, since Lemma 4.3 otherwise already yields the claim. As $D \leq R_{\alpha,L}$, and $R_{\alpha,L} \leq L/16$, this implies $\tau \leq 5L/8$.

The agents both take at most $2R_{\alpha,L} - 1 \leq L/8$ rounds from the start of their respective SearchingWalk to reach the node $r_{\alpha,L} = r_{\beta,L}$. Therefore, between rounds $T_{\alpha,L} + 4L + L/8$ and $T_{\alpha,L}+5L$, $\alpha$ is at $r_{\alpha,L} = r_{\beta,L}$. $\beta$ reaches this node by round $T_{\beta,L}+4L+L/8 \leq T_{\alpha,L}+\tau+4L+L/8 \leq T_{\alpha,L} + 4L + 3L/4$ at the latest. Thus the agents necessarily meet at $r_{\alpha,L} = r_{\beta,L}$ during the beginning of SearchingWalk before round $T_{\alpha,L} + 5L$. □

Finally, we tackle the last case of our analysis: the agents have computed nodes from the same ruling set, and they achieve rendezvous using the colors that were assigned to these nodes by ES-ColPathRulingSet.

**Lemma 4.7.** *Suppose that $R_{\alpha,L} = R_{\beta,L} \geq D$ and $r_{\alpha,L} \neq r_{\beta,L}$. Then $T_{\mathsf{rdv}} \leq T_{\alpha,L} + 28L \in O(L)$.*

*Proof.* Again let $\tau \leq 10D$ (Lemma 4.3), and note that $\tau \leq 10R_{\alpha,L} \leq L$.

Consider $c_{\alpha,L}$ and $c_{\beta,L}$ the colors assigned to the nodes $r_{\alpha,L}$ and $r_{\beta,L}$ by the execution of ES-ColPathRulingSet($R_{\alpha,L}$). As we have that $\mathrm{dist}(v_\alpha, r_{\alpha,L}) \leq 2R_{\alpha,L} - 1$, $\mathrm{dist}(v_\beta, r_{\beta,L}) \leq 2R_{\alpha,L} - 1$, and $\mathrm{dist}(v_\alpha, v_\beta) \leq D \leq R_{\alpha,L}$, it holds that nodes $r_{\alpha,L}$ and $r_{\beta,L}$ are at distance $\mathrm{dist}(r_{\alpha,L}, r_{\beta,L}) \leq 5R_{\alpha,L} - 2$. Algorithm ES-ColPathRulingSet($R$) computes a colored $(R, R-1)$-ruling set such that nodes of the ruling set get distinct colors when at distance $9R - 1$ or less from one another (Lemma 3.5). Therefore, $c_{\alpha,L} \neq c_{\beta,L}$.

Let $i$ be the first bit where $c_{\alpha,L}$ and $c_{\beta,L}$ differ. Let us assume that $\alpha$ has a 1 at index $i$ of its color while $\beta$ has a 0, the opposite case being similar. Between round $T_{\alpha,L}+5L+(2i+1)\cdot 32R_{\alpha,L}$ and $T_{\alpha,L}+5L+(2i+3)\cdot 32R_{\alpha,L}$, agent $\alpha$ is executing ZWalk($8R_{\alpha,L}$) twice in succession. In particular, $\alpha$ scans the entire distance-$8R_{\alpha,L}$ neighborhood of $r_{\alpha,L}$ between round $T_{\alpha,L}+5L+(2i+2)\cdot 32R_{\alpha,L}$ and $T_{\alpha,L} + 5L + (2i + 3) \cdot 32R_{\alpha,L}$. Recall that $\tau \leq 10D \leq 10R_{\alpha,L}$ and $T_{\beta,L} = T_{\alpha,L} + \tau$. Agent $\beta$ is staying at $r_{\beta,L}$ for $64R_{\alpha,L}$ rounds starting from round $T_{\beta,L} + 5L + (2i + 1) \cdot 32R_{\alpha,L} \leq T_{\alpha,L} + 5L + (2i + 1) \cdot 32R_{\alpha,L} + 10R_{\alpha,L}$. Therefore, $\alpha$ necessarily finds $\beta$ during its second ZWalk for bit $i$, by Proposition 3.2. See Fig. 3 for an illustration with $i = 3$ as first index where their colors differ. The bound on $T_{\mathsf{rdv}}$ simply follows from the runtime of SearchingWalk. □

## 4.5 Full Analysis

We now have analyzed all the cases necessary to show Theorem 1.1.

**Theorem 1.1.** *There is a deterministic algorithm for solving rendezvous on the infinite labeled line in $O(D \log^* \ell_{\min})$ rounds from the wake-up time of the first agent, where $D$ is the initial distance between the two agents and $\ell_{\min}$ is the smallest label within distance $O(D)$ from the agents' starting locations.*

*Proof.* Recall that $T_{\alpha,L} \in O(L)$ (Proposition 4.1). If the delay $\tau$ between the startup times of the two agents is $10D$ or larger, by Lemma 4.3, the agents reach rendezvous in $T_{\alpha,O(D)}+O(D) \in O(D)$ rounds ($T_{\mathsf{rdv}} \in O(D)$). We assume a smaller delay $\tau$ in what follows.

By Lemma 4.4, we know that $R_{\alpha,L} \geq D$ and $R_{\beta,L} \geq D$ for $L > \Theta(D \log^* \ell_{\min})$. For $L$ a large enough $\Theta(D \log^* \ell_{\min})$, if during the loop iteration of parameter $L$ the agents settle on different values $R_{\alpha,L} \neq R_{\beta,L}$ for their ruling sets' distances, they achieve rendezvous before round $T_{\alpha,L} + O(L) = O(L) = O(D \log^* \ell_{\min})$ by Lemma 4.5. If they have the same value $R_{\alpha,L} = R_{\beta,L}$, then they also achieve rendezvous before round $T_{\alpha,L} + O(L) = O(L) = O(D \log^* \ell_{\min})$, whether they go to the same ruling set node (Lemma 4.6) or different ones (Lemma 4.7). □

# 5 Corollary for Finite Graphs

In this section, we give simple extensions of our result for the infinite line to analogous path-like graphs like finite paths and cycles. Adapting our algorithm to also handle these graphs is only the matter of introducing some simple new behavior when an agent detects that the graph is finite (e.g., when it finds a node of degree 1 for a finite path). We prove Corollary 1.2.

**Corollary 1.2.** *There is a deterministic algorithm for solving rendezvous on finite labeled paths and cycles in $O(\min(n, D \log^* \ell_{\min}))$ rounds from the wake-up time of the first agent, where $D$ is the initial distance between the two agents, $\ell_{\min}$ is the smallest label within distance $O(D)$ from the agents' starting locations, and $n$ is the number of nodes in the graph.*

*Proof.* We do the following adjustment to our algorithm for the infinite line:

1. While an agent has neither seen a node of degree 1 (the end of a finite path), or reached the same node from two distinct edge-disjoint paths from its starting position (indicating a cycle), it simply executes the algorithm for the infinite line.

2. If an agent finds itself at a node of degree 1, it walks straight in the direction away from that node (using CarefulWalk)

3. If an agent detected that it is in a cycle, it goes to the node of minimum label in the graph, and waits there.

Suppose $n \in O(D \log^* \ell_{\min})$. After $\Theta(\min(n, D \log^* \ell_{\min}))$ rounds, if rendezvous has not yet occurred, the two agents have discovered the entire graph. In particular, they are both awake. For the finite line, this already implies that the agents achieved rendezvous by Proposition 3.2. For the cycle, the agents are guaranteed to meet once they have both reached the node of minimum ID, which must occur after $\Theta(\min(n, D \log^* \ell_{\min}))$ rounds (both agents must be awake by the time one of the two agents detects the cycle).

If $n \geq c \cdot D \log^* \ell_{\min}$ for a sufficiently large constant $c$, then the part of the graph that is explored by the two agents in $(c/8) \cdot D \log^* \ell_{\min}$ is indistinguishable from a segment from an infinite line. In that case, the agents meet in $O(D \log^* \ell_{\min})$ rounds from our analysis for the infinite line. □

On finite lines and the infinite half-line, the time to rendezvous also decreases with the distance of the agents to the nearest endpoint (node of degree 1) of the line.

*Remark* 5.1. On the finite labeled line and the infinite labeled half-line, rendezvous can be achieved in $O(\min(d + D, D \log^* \ell_{\min}))$ rounds, where $d = \min_{u \in \deg^{-1}(1)} \min_{v \in \{v_\alpha, v_\beta\}} \mathrm{dist}(u, v)$ is the minimum distance between the agents' starting locations and the graph's endpoint(s).

## Acknowledgments

We thank Sebastian Brandt for interesting discussions and anonymous reviewers for their valuable feedback. This research was conducted while Alexandre Nolin was at the CISPA Helmholtz Center for Information Security.

## References

[AG03] Steve Alpern and Shmuel Gal. *The Theory of Search Games and Rendezvous*, volume 55 of *International series in operations research and management science*. Springer, 2003. doi:10.1007/b100809.

[Alp95] Steve Alpern. The rendezvous search problem. *SIAM Journal on Control and Optimization*, 33(3):673–683, 1995. doi:10.1137/S0363012993249195.

[BG98] Vic Baston and Shmuel Gal. Rendezvous on the line when the players' initial distance is given by an unknown probability distribution. *SIAM Journal on Control and Optimization*, 36(6):1880–1889, 1998. doi:10.1137/S0363012996314130.

[BP24] Subhash Bhagat and Andrzej Pelc. Deterministic rendezvous in infinite trees. *Theoretical Computer Science*, 984:114313, 2024. doi:10.1016/j.tcs.2023.114313.

[CCG+11] Andrew Collins, Jurek Czyzowicz, Leszek Gasieniec, Adrian Kosowski, and Russell A. Martin. Synchronous rendezvous for location-aware agents. In *Proc. Int. Symp. on Distributed Computing (DISC)*, volume 6950 of *LNCS*, pages 447–459. Springer, 2011. doi:10.1007/978-3-642-24100-0_42.

[CV86] Richard Cole and Uzi Vishkin. Deterministic coin tossing with applications to optimal parallel list ranking. *Information and Control*, 70(1):32–53, 1986. doi:10.1016/S0019-9958(86)80023-7.

[DFKP06] Anders Dessmark, Pierre Fraigniaud, Dariusz R. Kowalski, and Andrzej Pelc. Deterministic rendezvous in graphs. *Algorithmica*, 46(1):69–96, 2006. doi:10.1007/s00453-006-0074-2.

[DPP23] Yoann Dieudonné, Andrzej Pelc, and Franck Petit. Almost universal anonymous rendezvous in the plane. *Algorithmica*, 85(10):3110–3143, 2023. doi:10.1007/s00453-023-01122-2.

[GP25] Younan Gao and Andrzej Pelc. Sniffing helps to meet: Deterministic rendezvous of anonymous agents in the grid. *Theoretical Computer Science*, 1050:115395, 2025. doi:10.1016/j.tcs.2025.115395.

[HS20] Juho Hirvonen and Jukka Suomela. *Distributed Algorithms 2020*. Online book, 2020. Accessed on Feb 8, 2025. Source code at github.com/suomela/da2020. URL: https://jukkasuomela.fi/da2020/.

[KSSK03] Evangelos Kranakis, Nicola Santoro, Cindy Sawchuk, and Danny Krizanc. Mobile agent rendezvous in a ring. In *Proc. IEEE Int. Conf. on Distributed Computing Systems (ICDCS)*, pages 592–599. IEEE, 2003. doi:10.1109/ICDCS.2003.1203510.

[KSV13] Amos Korman, Jean-Sébastien Sereni, and Laurent Viennot. Toward more localized local algorithms: removing assumptions concerning global knowledge. *Distributed Computing*, 26(5-6):289–308, 2013. doi:10.1007/s00446-012-0174-8.

[LA96] Wei Shi Lim and Steve Alpern. Minimax rendezvous on the line. *SIAM Journal on Control and Optimization*, 34(5):1650–1665, 1996. doi:10.1137/s036301299427816x.

[Lin92] Nathan Linial. Locality in distributed graph algorithms. *SIAM Journal on Computing*, 21(1):193–201, 1992. doi:10.1137/0221015.

[LS14] Juhana Laurinharju and Jukka Suomela. Brief announcement: Linial's lower bound made easy. In *Proc. ACM Symp. on Principles of Distributed Computing (PODC)*, pages 377–378. ACM, 2014. doi:10.1145/2611462.2611505.

[MGK+06] Gianluca De Marco, Luisa Gargano, Evangelos Kranakis, Danny Krizanc, Andrzej Pelc, and Ugo Vaccaro. Asynchronous deterministic rendezvous in graphs. *Theoretical Computer Science*, 355(3):315–326, 2006. doi:10.1016/j.tcs.2005.12.016.

[MP25] Avery Miller and Andrzej Pelc. Fast deterministic rendezvous in labeled lines. *Distributed Computing*, 38(4):435–489, 2025. doi:10.1007/s00446-025-00489-5.

[Pel00] David Peleg. *Distributed Computing: A Locality-Sensitive Approach*. SIAM, 2000. doi:10.1137/1.9780898719772.

[Pel19] Andrzej Pelc. Deterministic rendezvous algorithms. In Paola Flocchini, Giuseppe Prencipe, and Nicola Santoro, editors, *Distributed Computing by Mobile Entities, Current Research in Moving and Computing*, volume 11340 of *LNCS*, pages 423–454. Springer, 2019. Also at arxiv:2303.10391. doi:10.1007/978-3-030-11072-7_17.

[Sch81] Thomas C. Schelling. *The Strategy of Conflict*. Harvard University Press, 1981. First published in 1960. URL: https://www.hup.harvard.edu/books/9780674840317.

[TZ14] Amnon Ta-Shma and Uri Zwick. Deterministic rendezvous, treasure hunts, and strongly universal exploration sequences. *ACM Transactions on Algorithms*, 10(3):12:1–12:15, 2014. doi:10.1145/2601068.